

status: proposed date: 2026-05-16 tags: [pipeline, performance, ssd-wear, fseventsd, acceptance, build] supersedes: [] depends-on: [ADR-0025, ADR-0038, ADR-0039, ADR-0048, ADR-0150] implements: []

Minimise file-copy churn across the release pipeline

Context and Problem Statement

`npm run release` performs ~192k–260k file write events per release on a no-source-change rerun. On an M5 Max MacBook Pro (36 GB, APFS) this is load-bearing on two axes the user cares about more than wall time:

- 1. **SSD endurance.** Repeated release cycles during a heavy ADR-0181 wire-up loop (~25 releases/day across a multi-day council session) accumulate hundreds of millions of NAND-cell write events against finite flash lifespan. The user has explicitly flagged this as the primary perf concern; wall-time wins are secondary.
- 2. **fseventsd kernel buffer accumulation.** `fseventsd` records every file event in its in-memory ring; observed RSS reached 10.7 GB before manual flush on this machine. Spotlight was excluded for `/tmp/ruflo-*` via the marker landed in commit 36b1b55 , but a marker only suppresses Spotlight indexing — it does not suppress `fseventsd` event-record creation. Every file under `/tmp` still generates an event entry.

The 2026-05-16 analyst pass quantified the per-release write surface:

Phase	Files written / read	Persistence
copy-source rsync (5 forks, <code>-a --delete</code> with PROTECT filters)	~105k stats, mtime-syncs delta only	<code>/tmp/ruflo-build</code> persistent (16 GB)
codemod (32,592 files scanned, 4,782 writes; 0 cache hits last release)	~4.8k writes	in <code>/tmp/ruflo-build</code>
build (tsc emit, ~7–10k dist files; incremental works)	~7–10k writes	in <code>/tmp/ruflo-build</code>
publish-verdaccio (~40 <code>npm publish</code> , ~80 tarballs, ~160 MB)	~80 tarballs/release	<code>~/.local/share/verdaccio/storage</code> (19 GB, no GC)
acceptance: main install (<code>/tmp/ruflo-accept-XXXX</code> , fresh <code>mktemp -d</code>)	~58k files / ~2.0 GB	per-release, in cleanup trap
acceptance: wrapper-solo install (<code>/tmp/ruflo-wrapper-solo-XXXX</code> , fresh <code>mktemp -d</code> + isolated <code>--cache</code>)	~59k files / ~1.7 GB	per-release, not in cleanup trap — orphans on interrupt
acceptance: P5 background init (<code>/tmp/ruflo-p5-XXXX</code>)	~338 files	per-release, not in cleanup trap — orphans every run
acceptance: e2e snapshot (<code>cp -r "\$ACCEPT_TEMP/. " "\$E2E_DIR/"</code>)	~58k file copies	per-release, in cleanup trap
acceptance: <code>_e2e_isolate</code> (173 call-sites, ~15 file copies each)	~3k file copies	inside <code>E2E_DIR</code>
acceptance: per-check <code>mktemp -d /tmp/...</code> (91 patterns, ~40 do their own <code>init --full</code>)	~13.5k files	per-check, individual <code>rm -rf</code> — leaks on bash error

Phase	Files written / read	Persistence
acceptance: persistent npx cache	+1–2k files	/tmp/ruflo-accept-npxcache (37 GB, no LRU prune)
Per-release total (acceptance + pipeline)	~192k–260k file writes	mixed

The dominant *re-execution* contributors are the two npm installs (~~117k files combined~~) and the ~~e2e snapshot duplication~~ (58k file copies). The dominant *orphan* contributors are wrapper-solo dirs (4 of the 5 largest leftovers on /tmp today are 2.6 GB wrapper-solo orphans), per-check mktemp -d /tmp/... dirs (11 leftover ruflo-adr0100-*, 5 leftover ruflo-adr0177-*), and the unbounded npx cache (37 GB).

Recent perf work landed (build phase already largely optimised):

- 36b1b55 — Spotlight marker on /tmp/ruflo-*
- 3db6ea463 (forks/ruflo) — onnx-warmer kills npx @latest in init.ts
- d86bea2 — _HEAVY_CHECK_IDS skip set
- 20730d2 — node_modules cache reverted (reconciliation cost > fresh install at the main-install layer)
- 212d67c — reverted --ignore-scripts on cache-restored install (proves at least one @sparkleideas/* package has a non-idempotent postinstall — load-bearing for L3 below)
- b3d556f → bd5d878 — "share wrapper-solo install across 3 wrapper-install checks" landed 2026-05-07 02:21, reverted 2026-05-07 02:24 with no recorded reason. Load-bearing prior-art for L1 below; bisecting the revert trigger is a hard prerequisite to re-attempting
- 6e4da40 — incremental tsc (rsync excludes stale .tsbuildinfo)
- aa1de25 — test pinning the new contract

Latest run (r11): 669/0/9, acceptance total 308s (–18.6s vs baseline). Wall-time progress is real; the SSD/fseventsd surface has barely been addressed.

Decision Drivers

- SSD wear is the primary cost.** Reducing file events, not wall time, is the success metric.
- fseventsd accumulation scales with event count regardless of Spotlight markers.** Every file write under /tmp generates an event record; the marker only suppresses Spotlight content-indexing, not the kernel event ring.
- feedback-no-fallbacks / feedback-data-loss-zero-tolerance** . Any cross-release reuse must fail loudly when stale state corrupts a check. Better to invalidate-and-reinstall than to silently produce false-pass results.
- CLAUDE.md "THREE COMMANDS" boundary stays inviolate.** Changes here must not change the user-facing entry points (npm run test:unit, bash scripts/test-acceptance-fast.sh --group <group>, npm run release).
- Recent revert (20730d2 , node_modules cache) is informative, not blocking.** The revert proved that caching only the main install breaks even on wall time. This ADR proposes broader cache surface (main + wrapper-solo + e2e snapshot) so the freed time pays for the reconciliation cost.
- Build-phase optimisations already landed.** copy-source rsync, codemod hash cache, and --incremental tsc cover the build half adequately. This ADR's primary scope is the acceptance half (which writes ~3× as many files per release as the build half).
- forks/ changes minimised, not forbidden.** Fork production code is out of scope for pipeline optimisation; L13's fork-side test additions (where the function under test lives) are an explicit, documented exception. Default writes land in ruflo-patch (scripts/ , lib/).

Considered Options

- Option A — Status quo + Spotlight marker only.** Continue with mktemp -d per phase, rely on cleanup traps, accept the 192k–260k event surface. Rejected: doesn't address the primary cost.
- Option B — Reduce file-copy churn end-to-end (chosen).** Persistent install dirs (main + wrapper-solo merged or shared), copy-on-write e2e snapshot, per-check workdirs reparented under \$ACCEPT_TEMP, npx cache LRU pruning, Verdaccio storage

GC, stale-dir startup sweep widened, cleanup-trap coverage extended. Each lever is independent and ship-able in isolation.

- **Option C — Move the whole pipeline onto a RAM disk (`hdiutil attach -nomount ram://`).** Eliminates SSD wear entirely for `/tmp` writes. Rejected: 36 GB host RAM is tight for a ~25 GB working set (build + accept + e2e + npx cache); RAM disk eviction breaks the persistent ONNX cache + npx cache contracts (ADR-0048, ADR-0025); the marker-style mitigation is per-process and per-release while a RAM disk is per-host and survives across user sessions.
- **Option D — Run the pipeline in a Docker / Lima VM with its own filesystem.** Same goal as C with isolation. Rejected: doubles the trust surface (VM filesystem + macOS host filesystem still both see file events at the host layer); `napi build` cross-compilation matrix becomes brittle; out of scope as a release-time concern.
- **Option E — Switch `/tmp` → tmpfs-style mount.** macOS doesn't ship a real tmpfs; `/tmp` is already on APFS. Rejected as not architecturally available.
- **Option F — Replace remaining `cp` sites with `rsync` instead of `cp -cR (L2)`.** Rejected. `rsync` 's strength is incremental delta-update against an existing baseline, but every remaining `cp` site in the pipeline operates on a FRESH destination (mktemp'd per release). With no baseline to diff against, `rsync` reads + writes every file in user space — same I/O profile as `cp -r`, plus per-file compare overhead. Critically, `rsync` does not support reflink / `clonefile(2)`, so it bypasses APFS's CoW semantics entirely. Concrete e2e-snapshot comparison: `cp -r` reads 58k files × ~35 KB = ~2 GB read + ~2 GB written to flash, ~6-10s wall. `cp -cR` reads 58k inodes only + writes ~3 MB inode-table updates, ~200 ms wall. `rsync` would compare 58k file sizes/mtimes against an empty dest (every file flagged "new") then read + write ~2 GB, ~5-8s wall — strictly worse than `cp -cR` and matches `cp -r` on bytes-on-flash. The intuition "rsync to avoid copying" only holds when there is a stable destination baseline to diff against; that condition does not exist at any remaining `cp` site in this pipeline. (Where `rsync` DOES help — `scripts/copy-source.sh` 's 5 parallel fork → `/tmp/ruflo-build` syncs — it is already in use; see ADR-0038.)

Decision Outcome

Chosen option: **Option B — reduce file-copy churn end-to-end with four large levers + eight smaller levers (five hygiene + three test-corpus)**, prioritised by expected SSD-write-byte reduction per release. Each lever is implementable and verifiable in isolation; none of them requires touching the canonical user-facing entry points or `forks/`.

Reviewer note (DA round 1, 2026-05-16): an earlier draft included an "L5 — codemod content-equality short-circuit" lever projecting ~1.5-3k saved writes/release. DA grepped `scripts/codemod.mjs` and found the short-circuit is already implemented (`if (transformed !== content) { writeFile(...)` } at line 609; the `package.json` branch at L562-585 likewise reads → parses → only writes on `changed`). The lever was dropped; total projected wins adjusted downward accordingly.

Levers (prioritised by SSD-write-byte reduction)

#	Lever	Writes / bytes reduced per release	Risk	Mitigation
L1	Drop wrapper solo dedicated install; verify against main \$ACCEPT_TEMP DROPPED 2026-05-16	(forgone: ~59k writes + ~1.7 GB + ~70s wall — see "why dropped" below)	The original 2026-05-07 attempt (<code>b3d556f</code> , reverted 9 min later as <code>bd5d878</code>) had an unrecovered failure mode. Empirical retry under <code>RUFLO_SHARED_WRAPPER_INSTALL=1</code> (commit <code>8b9de92</code> , reverted as <code>a903c9c</code>) produced 62 acceptance failures vs 0 in baseline-1d-CANONICAL and 1085s duration vs 384s — see <code>logs/release-disk-bytes.jsonl</code> entry L1-experiment-RUFLO_SHARED_WRAPPER_INSTALL . Documented trigger (the bisect couldn't find from history alone): when <code>npm install @sparkleideas/ruflo</code> runs AFTER <code>ruflo init --full</code> against the same <code>\$ACCEPT_TEMP/node_modules</code> ,	DROPPED per override path - verified, it conf unworkable. Fi: rejected: (a) in changes accep test wrapper's the point of se vs @sparkleide install ruflo into \$ACCEPT_TEM WRAPPER_SO cost; (c) live w unacceptable. savings are fo ~70s wall cost commit <code>081a8</code> runs in backgr

#	Lever	Writes / bytes reduced per release	Risk	Mitigation
			the wrapper's pinned <code>@sparkleideas/cli</code> version mutates the <code>node_modules</code> in ways the init's already-resolved artifact paths no longer match. Concrete failure modes observed: (i) <code>adr0074</code> group — "intelligence.cjs not found in init'd project" / "auto-memory-hook.mjs not found in init'd project" (init's resolved paths stale after npm install); (ii) <code>sec-rl-consumed</code> — RVF error <code>0x0300 LockHeld</code> after 30s budget (daemon from cli's first install holds RVF lock; wrapper's separate cli version tries to open same file and deadlocks).	parallel wave — file-event orph covered indepe trap-cover WR L1 impact after regression, AD as the docume implementer w win" knows wh
L2	e2e snapshot via APFS clonefile (<code>cp -cR</code>) instead of <code>cp -r</code>	~2.0 GB byte-copies avoided (event-count win is smaller than first claimed; see Confirmation §Metrics)	<code>cp -c</code> is APFS-only; macOS <code>cp -c</code> is documented as non-standard; clone breaks on cross-volume operations. Earlier "1 clone op" framing was wrong — <code>cp -cR</code> invokes <code>clonefile(2)</code> per regular file, so <code>fs_usage</code> still reports ~58k clonefile events (the win is in <i>bytes written</i> , not events)	Feature-probe not <code>/dev/null</code> refuses — see <code>_cow_src=\$(mktemp -u _cow_dst=\$(mktemp -u dst.XXXX); cp -c _cow_dst" 2 HAS_COW=1; rm "\$_cow_dst" . Linux/foreign-F Each branch lo macOS version probe. Primary SSD-write-by</code>
L3	Persistent <code>ACCEPT_TEMP</code> with per-release reconcile (re-use <code>node_modules</code> across releases when CLI version + lockfile hash unchanged)	~58k writes / ~2.0 GB when cache-hit (most no-source-change reruns)	The <code>20730d2</code> revert + the earlier <code>212d67c</code> revert (" --ignore-scripts on cache-restored install") proved that the failure mode is npm reconciliation cost + postinstall idempotency , not cache-key breadth. Re-attempting this lever has to address the underlying disease, not the cache scope	Hard requirem skip npm inst run it against tl (that's the faile Cache key = SI <code>@sparkleidea</code> resolved <code>@spa</code> + <code>@sparkleide</code> package-lock < invalidates the (iii) Postinstall test must provi <code>node_modules</code> is byte-identici path and a cac produces non-rejected. (iv) E <code>ACCEPT_TEMP</code> epoch file con hash; harness matches the fr

#	Lever	Writes / bytes reduced per release	Risk	Mitigation
				nuke-fresh or (covers crasher Lands behind RUFLO_PERSISTENT switch: if not present 4 weeks of land resolution code)
L4	Reparent per-check mktemp -d /tmp/... patterns under \$ACCEPT_TEMP/_check_workdirs/\${id}-XXXX (LANDED 2026-05-16, commit dddd516 ; fallout fixes aab7276 + c81c9b9)	~13.5k writes/release move from /tmp to ACCEPT_TEMP (already covered by cleanup trap) + eliminates 91 distinct orphan sources	LANDED: 42 files modified (39 lib/ + 3 scripts/), 91 mktemp callsites reparented in a single mechanical sed pass. Empirical correction to the original draft: only 1 hard-coded /tmp/ site needed a by-hand fix (lib/acceptance-adr0117-marketplace-mcp.sh:208), not 6-8 — the original estimate was conservative. Parent-mkdir landed in 3 entry points, not 2: scripts/test-acceptance.sh , scripts/test-acceptance-fast.sh , and scripts/run-check.sh (run-check.sh was missed in the original draft). Documented exclusions extended beyond the original 4 persistent-cache list: ruflo-fast-* (ACCEPT_TEMP variant in fast runner), ruflo-e2e-* (E2E_DIR — separately trap-covered), ruflo-rsync-* (build copy-source), ruflo-promote-* (promote pipeline), plus 2 mktemp -d -t adr0069bug3-... callsites using the -t form (OS tmpdir, intentional per inline comment). Reparent template uses \${ACCEPT_TEMP:-/tmp} so ad-hoc check invocations (no harness wrapper) fall back to /tmp — and ad-hoc invocations don't have the trap-coverage promise anyway. TWO Wave-1 fallout patterns surfaced and were fixed: (a) Over-reach (commit aab7276): the agent's grep matched mktemp /tmp/ broadly, sweeping 37 FILE-create lines (mktemp /tmp/<id>-XXXXX without -d) along with the 91 DIR-create lines. File-creates in helpers (_run_and_kill / _run_and_kill_ro at acceptance-checks.sh:67,151 ; ~20 others in check-specific helpers) auto-clean inside the helper — they aren't orphan-on-trap concerns, and	

#	Lever	Writes / bytes reduced per release	Risk	Mitigation
			reparenting broke unit tests that source helpers outside the harness (no <code>_check_workdirs/ mkdir</code> runs). Future implementer prompts MUST explicitly enforce the <code>mktemp -d</code> boundary, not just <code>mktemp <path></code> . (b) Root-isolation fallout (commit c81c9b9): any check whose iso dir must be OUTSIDE any ruflo-project marker tree (<code>adr0100-A..G</code> for <code>findProjectRoot</code> walk-up tests; <code>adr0104-mcp-path</code> for <code>.mcp.json</code> write-to-self verification) breaks when reparented under <code>\$ACCEPT_TEMP/_check_workdirs/</code> because <code>\$ACCEPT_TEMP</code> IS a harness-init'd project. 8 callsites need a permanent <code>/tmp/</code> carve-out. Future L4-style sweeps must explicitly skip files containing checks that use <code>findProjectRoot</code> walk-up or test init's project-resolution behaviour. This is documented in the patch commit and now in this lever row	

Eight smaller levers (worthwhile but separately scoped):

#	Lever	Files reduced	Notes
L6	Register <code>WRAPPER_SOLO_TEMP + _P5_DIR</code> in cleanup trap	Eliminates ~2 orphan classes	2-line patch in <code>test-acceptance.sh:119-172</code> . Ships <i>first</i> — provides orphan-cleanup safety even if L1 is dropped after the b3d556f bisect
L7	Widen stale-sweep glob at startup (extend <code>find /tmp -maxdepth 1 -name "ruflo-accept-*"</code> to also match <code>ruflo-wrapper-solo-*</code> , <code>ruflo-p5-*</code> , <code>iso-*</code> , <code>ruflo-adr0*-*</code> , <code>t3*-rvf-*</code> , <code>b2*-work-*</code> , <code>p9*-*</code>)	Eliminates leftover-dir accumulation across crashed releases	<code>test-acceptance.sh:198</code>
L8	Verdaccio storage GC: keep last N tarballs per package	Caps <code>~/.local/share/verdaccio/storage</code> at ~700 MB (from 14 GB)	New script <code>scripts/verdaccio-gc.mjs</code> . Pre-GC check: enumerate all currently-pinned <code>-patch.N</code> refs in <code>forks/*/package.json</code> and the test corpus; refuse to drop a tarball that is still resolvable from any active source. Fires opportunistically at end of <code>ruflo-publish.sh</code>

#	Lever	Files reduced	Notes
L9	npx _cacache LRU prune (current sweep only clears _npx aliases)	Caps /tmp/ruflo-accept-npxcache at ~5 GB (from 37 GB)	Extend test-acceptance.sh:700
L10	Migrate remaining ~130 _e2e_isolate call-sites to _with_iso_cleanup RETURN-trap pattern (DEFERRED) + add fail-loud guard on the 4 source-existence checks in _e2e_isolate (LANDED 2026-05-16)	Eliminates leak surface on bash-error early-exit + closes the L3 cross-release symlink hazard	Source-existence guards LANDED in lib/acceptance-e2e-checks.sh via commits 2a4b8c8 (symlink line 29→34-38, using [[-d]] not realpath -e — BSD realpath on macOS lacks -e; [[-d]] follows symlinks and returns false for dangling/missing targets, subsuming both realpath -e failure modes portably) and 2e6e4a9 (3 cp swallows on .claude-flow / .swarm / package.json at lines 26-28 — same hazard class, surfaced by lever-L10 during Wave 1 review). Still DEFERRED: migrating the ~130 remaining _e2e_isolate call-sites to the _with_iso_cleanup RETURN-trap pattern; mechanical refactor in a separate follow-up. 41 sites already use that pattern
L11	Replace init-incident per-check inits with clone-from-golden (_e2e_isolate or _acceptance_snapshot) or .claude-flow/config.json mutation	LANDED 2026-05-16 commit 1117b66 : drops 2 of 17 per-release inits (~676 file writes + ~20-30s wall saved). Per-release total: 17 → 15.	The pipeline runs 17 distinct init --full invocations per release. Original projection: 3 clone-candidates (adr0104, hive-mind x 2) . Empirical correction during implementation: only 2 were actually clone-safe (hive-mind x 2 — adr0120 + adr0121 gossip/crdt agent-annotation greps). adr0104 was reclassified as init-as-SUT: check_adr0104_mcp_direct_path grep-asserts init's emitted .mcp.json content (mcpServers.ruflo.command == "npx" + args) — clone-from-golden would test the HARNESS'S emit, not the iso's, defeating the assertion. L11 + L13(b) together drop 4 of 17 inits (L11 drops 2 hive-mind, L13(b) drops 2 adr0177 default-mpnet) . The remaining 13 inits are either strict init-as-SUT or need init for other reasons (MiniLM model bytes not pre-warmed, etc.). Honours feedback-no-fallbacks : if a clone breaks because the check relied on init side-effects (daemon socket creation, fresh RVF seeding from cold), the check fails loud and runs its own init in a follow-up commit — no silent fallback. Lever lands <i>after</i> L2 since _e2e_isolate consumes L2's snapshot pattern
L12	Drop redundant bge model variant from adr0177	1 init removed per release (~338 file writes + ~10s wall time + bge ONNX download bytes on cache-miss only — steady-state ONNX cache per ADR-0048 already covers the model bytes)	Removes the _adr0177_run_init "\$dir" "Xenova/bge-base-en-v1.5" call at lib/acceptance-adr0177-checks.sh:298 and its scenario-2 assertion block. Coverage delta: bge-base-en-v1.5 and mpnet are both 768-dim with identical dim-validation,

#	Lever	Files reduced	Notes
			<code>.claude-flow/config.json</code> shape, RvfBackend segment dim, and <code>embeddings.json</code> runtime-read path. bge exercises no code path that mpnet doesn't also exercise. MiniLM × 1 stays under L12 (its removal is L13's concern). Tension with <code>feedback-no-value-judgements-on-features</code> : that memory forbids "redundant" as a gate for wire-vs-don't-wire decisions on <i>features</i>. L12 drops a <i>test variant</i> of an identical code path, not bge support itself — users can still pass <code>--embedding-model Xenova/bge-base-en-v1.5</code> and the runtime handles it. This distinction (drop test-of-feature ≠ drop feature) is surfaced here so a future audit can revisit whether the spirit of that memory should extend to test-corpus decisions
L13	Apply writer/reader test split to adr0177 specifically (embeddings end-to-end)	2–3 additional inits removed/release (adr0177's remaining 3 inits after L12 collapse to 0–1)	Generalises L11's "init-writer vs config-reader" principle to ADR-0177's config chain. Writer layer (fork-side; documented carve-out from §Decision Drivers): add <code>forks/ruflo/v3/@claude-flow/cli/__tests__/embedding-models.test.ts</code> exercising each of the 5 rows in <code>KNOWN_EMBEDDING_MODELS</code> (<code>Xenova/all-mpnet-base-v2</code> , <code>bge-base-en-v1.5</code> , <code>gte-base</code> , <code>all-MiniLM-L6-v2</code> , <code>all-MiniLM-L12-v2</code> → 768/768/768/384/384) against <code>validateEmbeddingModel()</code> and the <code>.claude-flow/config.json</code> + <code>.claude-flow/embeddings.json</code> emitter chain. Pure function tests, no <code>init --full</code> spawn. Reader layer (ruflo-patch acceptance, post-L11): clone golden once, mutate <code>.claude-flow/config.json</code> for the 384-dim case (one mutation; golden covers 768), assert RvfBackend creates a segment at the mutated dim and AgentDB round-trips a vector at that dim. Outcome: adr0177's remaining inits collapse from 3 (after L12) to 0–1 (0 if golden default coverage + the 384-mutation reader test suffice; 1 if an additional MiniLM-specific integration is kept for redundancy). Hard requirement before landing: fork-side writer tests must be present and passing — replacing acceptance inits with config-mutate without the writer-layer unit tests in place would silently move the dim-table validation off the test corpus. <code>feedback-no-fallbacks</code> honoured: failure of the writer-layer unit test must fail the fork-side build; failure of the reader-layer

#	Lever	Files reduced	Notes
			mutation test must fail the release acceptance run. No "best-effort" downgrade

L1–L4 are the load-bearing wins (~190k file-events of the ~192k total per-release surface, ~5.7 GB of byte-copies/installs). L6–L10 are hygiene improvements; L11–L13 are test-corpus refinements that drop ~6–8 of 17 per-release `init --full` invocations. Each is small, mechanical, and ships independently.

Out of scope for this ADR

- `cross-repo/ruvector/target/ rsync exclusion` (16 GB, 13 GB of which is Cargo `target/`). Pipeline-analyst flagged this as the largest single contributor on the build half. It's a build-half concern with its own risk profile (ruvector currently relies on `target/` cache survival for napi rebuild speed per ADR-0150) and is scoped to a separate follow-up.
- `.tsconfig.build.json generation cache` (40 node spawns/release). Build-half concern; see L5 of the codemod work for the precedent.
- `.npmignore / .gitignore unconditional find ... -delete sweep` in `build-packages.sh:45` . Build-half concern; trivial cache gate worth a separate small commit.

Consequences

- Good, because **SSD-write-bytes per no-source-change rerun drop by ~5.7 GB** (L1 1.7 GB install + L2 2.0 GB e2e snapshot bytes + L3 2.0 GB main install when cache-hit). This is the load-bearing win — file-event count and SSD-write-bytes are non-equivalent metrics (see Confirmation §Metrics).
- Good, because `fsevents` event-record count drops from ~192k/release to ~70k/release (L1 saves ~59k events directly; L2 retains its ~58k clonefile events but those are metadata-only; L3 saves ~58k when cache-hit; L4 reparents ~13.5k events from `/tmp` into `ACCEPT_TEMP` — they still register with `fsevents` but cluster under one trap-covered prefix).
- Good, because orphan-dir accumulation on `/tmp` (74 GB today) drops to near-zero via L6 + L7 + L10's symlink fail-loud.
- Good, because Verdaccio storage and npx cache stop growing unbounded (L8, L9).
- Good, because the 4 large + 8 small levers (5 hygiene + 3 test-corpus) are independently ship-able — partial landing still delivers proportional wins, and each can be rolled back if it surfaces a regression.
- Good, because **L11/L12/L13 reduce per-release `init --full` invocations from 17 → ~9–11** (L11 drops ~3 via clone-from-golden / config-mutate; L12 drops 1 via bge variant removal; L13 drops ~2–3 by applying the writer/reader split to `adr0177`). This pushes the per-release `init` surface to the architectural floor of 9 (the must-run inits where `init`'s own code IS the SUT — `adr0088` × 2 PATH-probing, `adr0098` × 3 prior-state, `adr0100` × 1 `cwd`, `adr0143-init-mcp` × 1 wrapper-bin, `phase10` × 1 idempotency, plus the golden harness × 1).
- Bad, because **L1 has unrecovered prior-art evidence**: `b3d556f` → `bd5d878` two-hour revert cycle on 2026-05-07 implemented essentially the same shared-wrapper-install idea and bounced. Re-landing without bisecting the revert trigger risks repeating the regression. The pre-flight bisect requirement in the L1 mitigation column is load-bearing, not optional.
- Bad, because **L3 has the heaviest correctness contract** — postinstall idempotency must be proven by integration test, not assumed. The `212d67c` revert ("revert `--ignore-scripts` on cache-restored install") proves at least one `@sparkleideas/*` package has a non-idempotent postinstall; that package must be identified and either fixed or excluded from L3's cache scope.
- Bad, because L2's APFS clonefile is platform-specific. On a foreign FS (NFS-mounted scratch dir, ZFS, ext4 in a Linux CI runner) the fallback to `cp -al` or `cp -r` kicks in. Each branch logs which path fired — silent regression is avoided. The fork's CI today runs only on the user's M5 Max so foreign-FS support is theoretical.
- Bad, because L4 reparents under `${ACCEPT_TEMP:-/tmp}` — when `ACCEPT_TEMP` is unset (ad-hoc check invocation outside the harness), the `workdir` lands at `/tmp/_check_workdirs/...` which the trap doesn't cover. Ad-hoc invocations were never trap-covered before either, but the patch must document this rather than imply universal coverage.
- Neutral, because the canonical entry points (`npm run release` , `bash scripts/test-acceptance-fast.sh --group <group>` , `npm run test:unit`) are unchanged. Users see no behaviour delta except faster runs.
- Neutral, because the `feedback-no-fallbacks / feedback-data-loss-zero-tolerance` rules are honoured: L1's bisect requirement names the failure mode before re-landing; L2's clonefile probe + branch-logging makes the fallback explicit; L3's epoch guard + integration test gate the cache; L10's symlink fail-loud replaces a silent `2>/dev/null || true` .

Confirmation

- **Gate.** `npm run release` is the only release-time gate. Each lever lands behind a feature env-var first (`RUFLO_SHARED_WRAPPER_INSTALL=1` , `RUFLO_E2E_CLONEFILE=1` , `RUFLO_PERSISTENT_ACCEPT=1`) so it can be A/B-compared against the baseline before becoming default.
- **Metric — SSD-write-bytes (primary).** SSD-write-bytes per release is the SSD-wear proxy the user actually cares about. Capture via `ioreg` on macOS 26.x (`iostat -K` and `diskutil info WriteBytes` documented in earlier drafts do not exist on this host — DA round 2 R1):

```
# pre-release
ioreg -c IOBlockStorageDriver -r \
| awk '/"Bytes \(\Write\)"/={gsub(/\^[0-9]/,"",$0); print; exit}' \
> /tmp/ioreg-pre

npm run release

# post-release; diff is the release's contribution to cumulative writes
ioreg -c IOBlockStorageDriver -r \
| awk '/"Bytes \(\Write\)"/={gsub(/\^[0-9]/,"",$0); print; exit}' \
> /tmp/ioreg-post
echo $(( $(cat /tmp/ioreg-post) - $(cat /tmp/ioreg-pre) )) >> logs/release-disk-bytes.jsonl
```

Append to `logs/release-disk-bytes.jsonl`. **Caveat:** `ioreg` cumulative counters include EVERY write to disk0 across all processes during the release window — concurrent browser/IDE/Spotlight writes land in the same bucket. The guard-band threshold below absorbs this. Optional higher-fidelity capture for forensic re-runs: `sudo fs_usage -w -f filesystems ./scripts/ruflo-publish.sh` gives per-syscall byte accounting but requires sudo and is high-overhead — not the default gate. Hard gate: SSD-write-bytes per release must not exceed $\max(\text{baseline} \times 1.10, \text{baseline} + 100 \text{ MB})$. The $+ 100 \text{ MB}$ floor absorbs unrelated host activity below which the metric is too noisy to be meaningful.

- **Metric — fsevents event count (secondary).** `scripts/measure-file-events.sh` uses `fs_usage -w -f filesystems ... | awk` to count write/clonefile events during a release. Append to `logs/file-events.jsonl`. **Soft gate (target, not hard fail):** event count should drop, but L2's clonefile count is by-design ~equal to the prior `cp -r` open/write count — the win is in *bytes* per event, not event count.
- **Metric — wall-time.** Existing `timing-report.mjs` already records per-phase wall. Each lever ships a net wall-time neutral-or-better delta in addition to the byte-reduction win.
- **Acceptance correctness invariant (hard gate).** `npm run release` must satisfy, vs the baseline measured at the lever's landing time:
 - Pass count must **not decrease**.
 - Fail count must remain **0**.
 - Skip count must **not increase**. Any lever that regresses any of those three is reverted. (Hard-coded "669/678" numbers drift; these three contracts don't.)
- **L1 pre-flight (hard requirement).** Before landing L1, bisect `b3d556f` vs `bd5d878` to identify which acceptance check failed and why. Cite the trigger in the L1 commit message. If the trigger cannot be identified, L1 is dropped.
- **L3 cross-release correctness (hard requirement).** Add `tests/integration/persistent-accept-cache.test.mjs` proving: (a) cache invalidates on `@sparkleideas/cli` version bump; (b) cache invalidates on `.release-epoch` mismatch; (c) `node_modules/@sparkleideas/*/dist/` is byte-identical between cache-hit and cache-miss paths (postinstall idempotency). Lever does not promote to default until all three pass.
- **Kill-switch (hard requirement for L1 and L3).** If `RUFLO_SHARED_WRAPPER_INSTALL=1` or `RUFLO_PERSISTENT_ACCEPT=1` is not promoted to default within 4 weeks of landing (commit date + 28 days), the env var, its code path, and the integration test are fully reverted. No orphan opt-in flags.

Pros and Cons of the Options

Option B (chosen) — four large + five hygiene levers

- Good, because each lever is ship-able in isolation; staged landing reduces blast radius.
- Good, because SSD-write-bytes is directly measurable via `ioreg -c IOBlockStorageDriver` deltas, so the load-bearing user-facing metric is not speculative.

- Good, because L1 and L3 carry explicit kill-switches (4-week promote-or-revert) so a stalled A/B experiment doesn't leave orphan opt-in flags.
- Bad, because L1's prior-art (b3d556f → bd5d878) is a documented failure of the exact same idea. The bisect requirement is a hard prerequisite, not a nice-to-have.
- Bad, because L3 reintroduces the cross-release state question that both 20730d2 and 212d67c reverts opted out of. The reverted approach failed on npm reconciliation cost + postinstall non-idempotency, not cache-key breadth. L3's integration-test gate must prove postinstall idempotency before promotion.
- Bad, because L4's \${ACCEPT_TEMP:~/tmp} fallback for ad-hoc invocations weakens the "trap-covered scope" invariant for one specific call path. Documented, not silenced.

Option C — RAM disk

- Good, because eliminates SSD wear entirely for /tmp events.
- Bad, because 36 GB host RAM is tight; the ~25 GB working set leaves <11 GB for the rest of the system + browsers + dev tools.
- Bad, because the persistent ONNX cache (~/.cache/agentdb-models , ADR-0048) and npx cache (/tmp/ruflo-accept-npxcache , ADR-0025) both rely on cross-release survival. Putting them on a RAM disk breaks both contracts.
- Bad, because fseventsd still records events for /tmp regardless of backing store — the kernel doesn't know the FS is in-memory. So RAM disk solves SSD wear but not fseventsd accumulation.

Option D — Docker / Lima VM

- Good, because full isolation.
- Bad, because doubles trust surface for napi build cross-compilation matrix.
- Bad, because host filesystem still sees mount events (does not fix fseventsd).

Option F — rsync everywhere instead of cp -cR (L2)

- Good, because conceptually appealing — "incremental copy" feels like the right answer for "minimise file copying."
- Bad, because every remaining cp site operates on a fresh mktemp destination. No baseline → no delta → rsync degrades to cp -r with compare overhead.
- Bad, because rsync has no reflink / clonefile(2) support. On APFS it bypasses CoW entirely and writes full file bytes to flash. cp -cR writes only inode-table updates (~50 B/file) and defers data writes until a side mutates.
- Bad, because the architecturally-correct rsync site is scripts/copy-source.sh (5 parallel fork→build delta syncs), and that's already in production via ADR-0038.

More Information

Test-suite design principle: separate init-writer from config-reader tests

L11's reducibility analysis surfaces an architectural principle that is currently *conflated* in the acceptance corpus: many checks use `init --full` as the easiest way to put a project into "config has value V" state, then assert downstream behaviour. That bundles two distinct assertions into one expensive setup:

1. " `init --full --embedding-model M` writes the correct values into `.claude-flow/config.json` " (the **writer** test — `init` is the SUT).
2. "Given `embedding.model = M`, `embedding.dimension = D` in config, the runtime / AgentDB / RvfBackend behaves correctly" (the **reader** test — config-and-runtime is the SUT; `init` is just one possible way to produce that config).

The current corpus mostly tests (1) by way of (2) — re-running `init` for each downstream variant. The split delivers two independent benefits:

- **Writer tests** become small, fast, and dim-of-the-table-driven. `validateEmbeddingModel()` is already a pure function over a 5-row frozen table (`forks/ruflo/v3/@claude-flow/cli/src/init/embedding-models.ts:14-22`); the writer test is "for each row, `init`'s config-emitter produces these bytes" — exactly the cardinality of the table, in a unit test, without spawning `init`.
- **Reader tests** become "mutate config, exercise the runtime, assert behaviour" — one `init` (the golden), N config mutations covering whichever (model, dim) tuples matter. Adding a new model becomes "add a row to the lookup table + add a writer-test

case + add a reader-test case using config-mutation" — no new `init --full` invocation per model.

The principle is broader than embeddings: any check whose name reads as "init writes X" belongs in the writer bucket (unit-test the config-emitter); any check whose name reads as "with X in config, behaviour Y" belongs in the reader bucket (clone-from-golden + config-mutate). The current acceptance corpus has 8 inits sitting on the wrong side of this line (adr0104 × 1, adr0177 × 4, adr0178 × 1, hive-mind × 2). L11 implements the separation for those 8 call sites.

Where the principle does NOT apply: checks where init's *execution under specific conditions* is what's being asserted (PATH stripping in adr0088, prior-state reactions in adr0098, cwd handling in adr0100, idempotency in phase10, wrapper-bin generation in adr0143-init-mcp). Those 9 checks legitimately keep their full init invocations — init's code IS the writer being tested, under conditions a unit test cannot reproduce.

Wave 1 landed (2026-05-16)

Five commits on `main` from a 4-agent parallel swarm:

Commit	Lever	Notes
bd467cb	L6	Trap-cover <code>WRAPPER_SOLO_TEMP + _P5_DIR</code> . All 5 top-level temp dirs now trap-covered
2a4b8c8	L10 (symlink guard)	Used <code>[[-d]]</code> instead of <code>realpath -e</code> (BSD <code>realpath</code> on macOS lacks <code>-e</code> ; <code>[[-d]]</code> follows symlinks and is the portable equivalent)
8d1a1cf	L7	Widen stale-sweep to 7 prefix classes via single multi-line <code>find ... \ (-name ... -o ... \)</code>
aed01d4	L12	Drop bge model variant. -77 LoC in <code>lib/acceptance-adr0177-checks.sh</code> ; 4 outside-file registry refs cleaned in <code>scripts/test-acceptance.sh + scripts/test-acceptance-fast.sh</code> . MiniLM scenario renumbered 3→2 (check IDs unchanged, log scrapers unaffected)
dddd516	L4	Reparent 91 mktemp workdirs across 42 files. 1 hard-coded site by-hand (not 6-8 as estimated). 3 entry-point makedirs (<code>test-acceptance.sh</code> , <code>test-acceptance-fast.sh</code> , <code>run-check.sh</code>)
2e6e4a9	L10 (cp guards)	Follow-up: same fail-loud pattern applied to 3 sibling <code>cp</code> swallows at <code>_e2e_isolate:26-28</code> (surfaced by lever-L10 during Wave 1 review)
aab7276	L4 over-reach fix	Reverts 37 file-create mktemp reparentings (<code>mktemp /tmp/<id> without -d</code>) — agent's grep matched broader than the lever spec. Foundation helpers (<code>_run_and_kill / _run_and_kill_ro</code> in <code>acceptance-checks.sh</code>) caught the regression first because unit tests source them outside the harness. All 3936 unit tests green post-fix
c81c9b9	L4 root-isolation fix	Reverts 8 mktemp reparentings for root-isolation checks: adr0100-A..G (<code>findProjectRoot</code> walk-up tests) and adr0104-mcp-path (<code>.mcp.json</code> write-to-iso verification). These checks require their iso to be OUTSIDE any ruflo-project marker tree; reparenting under <code>\$ACCEPT_TEMP/_check_workdirs/</code> placed them inside the harness's init'd project and broke the precondition. Surfaced by baseline-1 release run (<code>logs/release-disk-bytes.jsonl</code> : baseline-1, 18.67 GB writes, 378s, exit=1)
8b9de92 → a903c9c	L1 empirical retry → revert	User-directed retry of the share-install approach under <code>RUFLO_SHARED_WRAPPER_INSTALL=1</code> . Empirically confirmed unworkable: 62 acceptance failures vs 0 baseline; 1085s vs 384s duration. Trigger identified: <code>npm install @sparkleideas/ruflo</code> after <code>ruflo init --full</code> mutates <code>node_modules</code> in ways init's resolved artifact paths no longer match (adr0074 group: <code>intelligence.cjs</code> not found ; <code>sec-rl-consumed</code> : RVF LockHeld). L1 dropped permanently — see L1 row above for documented gravesite
adfd077	L2 LANDED	<code>_acceptance_snapshot()</code> helper at <code>lib/acceptance-harness.sh:96-143</code> ; COW probe with a real regular file; <code>cp -cR</code> fires on this host (macOS 26 / Darwin 25.4.0). E2E

Commit	Lever	Notes
		snapshot at <code>test-acceptance.sh:414</code> . Agent added <code>rmdir "\$E2E_DIR"</code> before helper call so all 3 branches (cow/hardlink/recursive) have identical dst-doesn't-exist semantics
8d50332	L8 LANDED	<code>scripts/verdaccio-gc.mjs</code> (24,759 bytes) with 4-source pin-ref enumeration (forks self/dep-map, codemod UNSCOPED_MAP, .last-build-state; .release-epoch deferred to L3). Hooked at <code>ruflo-publish.sh:468-478</code> ; GC failure doesn't fail release. Dry-run on this host: would-drop 21,503 of 22,134 versions across 64 packages, freeing 13.58 GiB
b64e11f	L9 LANDED	<code>_l9_lru_prune_npx_cache</code> at <code>test-acceptance.sh:733-794</code> . BSD <code>stat -f</code> works on this host; LRU sort by mtime ascending; pruned 238/247 blobs in dry-run at <code>RUFL0_NPX_CACHE_CAP_MB=1</code> . Current <code>_cacache</code> is 37 GB — will prune ~33 GB to 5 GB cap on next release startup
1117b66	L11 LANDED	2 hive-mind checks (adr0120, adr0121) converted to <code>_e2e_isolate clone-from-golden</code> . ADR's "3 candidates" was off by one (adr0104 reclassified as init-as-SUT). Fast-acceptance shows 93ms / 85ms (was ~10-15s); init count 17 → 15
2a9829d	L3 LANDED (opt-in)	<code>lib/accept-cache.sh</code> + <code>tests/integration/persistent-accept-cache.test.mjs</code> . Gated by <code>RUFL0_PERSISTENT_ACCEPT=1</code> . 6/6 integration tests pass including 62-second byte-identity proof of <code>node_modules/@sparkleideas/*/dist/</code> between cache-hit and cache-miss paths. Cache key uses cli's <code>dependencies</code> block as lock-equivalent (cli doesn't ship <code>package-lock.json</code> , verified against patch.161). Cache dir at <code>~/.cache/ruflo-accept-persistent-<hash></code> (Temp-Cleaner-safe). 4-week kill-switch: revert flag + code path by 2026-06-13 if not promoted to default
6055e04cc (fork)	L13(a) LANDED	<code>forks/ruflo/v3/@claude-flow/cli/__tests__/embedding-models.test.ts</code> — 25 vitest tests in 2ms, all 5 KNOWN_EMBEDDING_MODELS rows × <code>validateEmbeddingModel</code> + config-emitter round-trip + BARE_NAME/UNKNOWN_MODEL negatives. Runs in fork CI. On fork's <code>main</code> ; ships to Verdaccio on next release
34cbf58	L13(b) LANDED	adr0177's 2 default-mpnet inits converted to <code>_e2e_isolate clone-from-golden</code> . MiniLM scenario kept as real init (model bytes not pre-warmed). Init count 15 → 13
1117b66 → 1767ef8 (revert) → 47ee247 (cherry-pick) + d0d5cbe (snapshot fix)	L11 RE-LANDED 2026-05-17	First attempt (1117b66) tripped on <code>_acceptance_snapshot</code> returning non-zero when copying <code>\$E2E_DIR/.claude-flow/daemon.sock</code> (BSD <code>cp -cR</code> can't copy sockets); reverted as 1767ef8 . Re-attempt landed via cherry-pick 47ee247 + snapshot socket-tolerance fix in d0d5cbe : <code>_acceptance_snapshot</code> now redirects per-file cp stderr (sockets / wal / lock files) and returns 0 unconditionally — mirrors L10-ext's narrowing for volatile-file races (commit 32121db). Downstream callers verify their specific expected files via existence checks. Final init drop: 2 (adr0120 + adr0121 gossip/crdt agent-annotation checks)
34cbf58 → bb14830 (revert) → d0d5cbe (P5_DIR re-attempt)	L13(b) RE-LANDED 2026-05-17	First attempt (34cbf58) used <code>_e2e_isolate</code> which reads <code>\$E2E_DIR</code> . But adr0177 runs in Phase 5 — <code>\$E2E_DIR</code> is wiped at <code>scripts/test-acceptance.sh:3017</code> BEFORE Phase 5 starts (commit bb14830 reverted that attempt). Phase 5 has its own golden dir, <code>\$P5_DIR</code> , exported at line 3047 from <code>_P5_DIR</code> (a backgrounded init joined just before the Phase 5 wave). Re-attempt in d0d5cbe : <code>check_adr0177_default_config_keys</code> reads <code>\$P5_DIR/.claude-flow/config.json</code> directly (pure read, no iso needed); <code>check_adr0177_default_roundtrip</code> clones <code>\$P5_DIR</code> via <code>_acceptance_snapshot</code> into an iso then runs memory store/search/check-dim. Final init drop: 2 (adr0177's 2 default-mpnet inits collapsed; MiniLM scenario still uses real init because model bytes aren't pre-warmed)
f8d3479	L2 catastrophic fix	The original L2 helper (adfd077) used <code>cp -cR src dst</code> and required the caller to <code>rmdir \$dst</code> first. <code>_disable_spotlight_indexing</code> adds a marker file between mktemp and rmdir at <code>scripts/test-acceptance.sh:462</code> , breaking rmdir and causing <code>cp -cR src dst</code> to create <code>dst/\${basename src}</code> instead — landing the 2.1 GB snapshot at the

Commit	Lever	Notes
		wrong path. Wave 3 baseline showed 59 acceptance failures from this. Fix: switch helper to <code>cp -cR src/. dst/</code> form (contents-into-existing-dst, matching the prior <code>cp -r</code> behaviour). Verified on APFS: marker preserved, exit 0. Removed the <code>rmdir</code> hack from the caller

Implementation roadmap

A future implementer reads this ADR and lands levers in this order, each as a separate commit on `main` :

- L6** (cleanup trap registration) — 2-line patch, lowest risk, ships immediately. Provides orphan-cleanup safety even if L1 is later dropped.
- L7** (widened stale-sweep glob) — one-line `find` extension; ships immediately.
- L10** (iso symlink fail-loud guard) — change `lib/acceptance-e2e-checks.sh:29` from `ln -sf ... 2>/dev/null || true` to `realpath -e check + fail-loud`. Closes the cross-release symlink hazard before L3 lands.
- L4** (per-check workdir reparenting) — single mechanical regex pass `s|mktemp -d /tmp/<id>-XXXXX|mktemp -d "${ACCEPT_TEMP:-/tmp}/_check_workdirs/<id>-XXXXX"|g` over `lib/acceptance-*.sh`. The 6–8 hard-coded `/tmp/` paths are fixed by hand in a follow-up commit. No "audit" — just the sweep.
- L12** (drop bge model variant) — mechanical: delete `_adr0177_run_init "$dir" "Xenova/bge-base-en-v1.5"` call at `lib/acceptance-adr0177-checks.sh:298` and the scenario-2 assertion block. Single commit; ships independently of all other levers.
- L2** (e2e clonefile) — extract `_acceptance_snapshot()` helper that probes COW with a real regular file (NOT `/dev/null` — see DA round 1 B2), falls back to `cp -al`, then `cp -r`. Each branch logs which path fired. Lands in `test-acceptance.sh:385–387`.
- L11** (clone-from-golden / config-mutate for init-incidental checks) — uses L2's `_acceptance_snapshot()` helper to clone the golden post-init state per check. Per-check audit table goes in the commit body, tagging each of the 8 config-outcome inits as either CLONE (post-init state suffices), MUTATE (config-edit + read-path), or SUT (init's own code is the assertion). The 3 obvious clone candidates (`adr0104`, `hive-mind` × 2) land in the first commit; `adr0178` audit follows in a second commit. `adr0177`'s 3 remaining inits (after L12) are NOT touched by L11 — they move to L13 since they need writer-layer unit-test coverage to land safely. **Hard requirement:** the commit message tags each call site with its decision; no silent "left as-is" entries.
- L13** (writer/reader split for `adr0177` embeddings) — depends on L11's mutate infrastructure. Lands in two commits: **(a) fork-side writer tests** at `forks/ruflo/v3/@claude-flow/cli/_tests_/embedding-models.test.ts` covering all 5 rows of `KNOWN_EMBEDDING_MODELS` against `validateEmbeddingModel()` + the config-emitter chain; CI gate on these passing before (b). **(b) ruflo-patch-side reader tests** rewriting `lib/acceptance-adr0177-checks.sh` to clone golden + mutate `.claude-flow/config.json` for the 384-dim case; the 2 default-model inits collapse into golden coverage; `MiniLM` × 1 collapses into the 384-mutation reader test. **Hard requirement:** (a) lands and ships in a released CLI version before (b) lands — acceptance against an old CLI without the writer tests would silently move the dim-table validation off the test corpus.
- L1** (drop wrapper-solo install) — **prerequisite: bisect b3d556f / bd5d878 to identify the 2026-05-07 revert trigger** before any code lands. With the trigger documented, replace `mktemp -d "/tmp/ruflo-wrapper-solo-XXXXX"` block in `test-acceptance.sh:235` and re-target the 3 wrapper checks (`adr0142` bin-path, `adr0142` MCP JSON-RPC, `adr0143` init MCP) at `$ACCEPT_TEMP`. Lands behind `RUFLO_SHARED_WRAPPER_INSTALL=1`. 4-week kill-switch.
- L3** (persistent `ACCEPT_TEMP` with cache key) — new module `lib/accept-cache.sh` exporting `_resolve_accept_temp()`. Cache key SHA-256 over (resolved `@sparkleideas/cli` version + resolved `@sparkleideas/ruflo` version + `@sparkleideas/cli`'s shipped `package-lock.json` content). Writes `.release-epoch` to `ACCEPT_TEMP`. Startup verifies epoch matches; nukes-fresh on mismatch/missing. Integration test in `tests/integration/persistent-accept-cache.test.mjs` proves invalidation triggers + postinstall byte-identity. Lands behind `RUFLO_PERSISTENT_ACCEPT=1`. 4-week kill-switch.
- L8, L9** — hygiene levers; ship in any order after L1–L7.

Open questions for implementer

- Postinstall identity (L3).** Which `@sparkleideas/*` package was the trigger for `212d67c` (revert `--ignore-scripts` on cache-restored install)? Identifying it pre-implementation lets the L3 integration test target that package's postinstall specifically. If postinstall non-determinism cannot be characterised, L3's gate is unmeetable and the lever is rejected.

- **L1 revert trigger (bd5d878) — cold-trail policy.** The revert commit message says only "Revert b3d556f"; the original lasted ~2 hours on `main` before bouncing. `git reflog` may surface the trigger; failing that, the `081a8f5` ("background install, join before wave") commit hints — perhaps the install was not in fact "stateless wrt install dir" as the original commit claimed. **Policy if the trail goes cold (DA round 2 answer):**
 - **Default — drop L1 entirely.** The wrapper-solo install costs ~70s wall (already backgrounded; net wall ≈ 0) and ~59k file events. Orphan damage is covered by L6 cleanup-trap registration anyway. L1's headline win was the 1.7 GB SSD-write reduction — losing it isn't catastrophic.
 - **Override path — rare and explicit.** If the implementer can articulate a specific testable hypothesis for why `bd5d878` reverted (e.g. "wrapper-check N depends on a postinstall script that didn't run because npm cache hit"), and the hypothesis is verified in isolation BEFORE landing, then ship L1 behind the opt-in with the kill-switch. The hypothesis goes in the commit message — no silent "we don't know, let's try anyway."
 - **Why not "best-effort + opt-in" as default:** that pattern is exactly the `feedback-no-fallbacks` shape DA explicitly named — un-named failure modes are the ones that catch you. Opt-in flags buy time but do not protect against unknown regressions; the 4-week kill-switch is the bisect's safety net, not its replacement.
- **L2 clonefile semantics under subsequent writes.** `cp -cR` on APFS produces a CoW clone — subsequent writes to the destination tree allocate new blocks lazily. Worth verifying that `_e2e_isolate 's rm -f "$iso_dir/.swarm/memory.rvf"*` (line 31) doesn't accidentally drag the underlying source blocks into the snapshot.
- **L11 per-check audit table.** For each of the 8 config-outcome inits (`adr0104` $\times$ 1, `adr0177` $\times$ 4, `adr0178` $\times$ 1, `hive-mind` $\times$ 2), the implementer must tag each call site CLONE (clone-from-golden suffices), MUTATE (clone + `.claude-flow/config.json` edit), or SUT-AFTER-ALL (init's code is the assertion target despite first impressions). The 3 obvious clone candidates (`adr0104`, `hive-mind` $\times$ 2) land first; the remaining 5 (`adr0177`'s 2 defaults + 1 bge [now L12'd] + 1 MiniLM, plus `adr0178`'s `--with-embeddings`) need by-hand reading of the assertion blocks. Empirical evidence to ground the audit: `forks/ruflo/v3/@claude-flow/cli/src/init/embedding-models.ts:14-22` confirms `KNOWN_EMBEDDING_MODELS` is a static frozen lookup table (3 $\times$ 768-dim + 2 $\times$ 384-dim entries) with no runtime dim-reduction logic, so "dim auto-adjustment" reduces to "writer looks up model's dim and emits it into config" — a writer-test concern, not a reader-test concern. MiniLM $\times$ 1's reader-bucket assertions (runtime can create a 384-dim segment, AgentDB round-trips at 384-dim) could be re-expressed as a config-mutate + assert against golden, which would push the audit's reduction count from 3 $\rightarrow$ 5+.
- **L8 (Verdaccio GC) pre-check correctness.** Enumerate every source of pinned `-patch.N` refs before deciding which tarballs to drop. If a tarball is referenced from any of the four sources below, NEVER drop it regardless of age. Keep-last-N (default `N=10`) is the floor; the enumeration is the ceiling.
 - (a) `forks/*/package.json` — internal cross-fork deps.
 - (b) `scripts/codemod.mjs` 's rewrite tables — every `-patch.N` ref the codemod generates is load-bearing for the codemod's expected output.
 - (c) `.last-build-state` — references the resolved versions at last successful release.
 - (d) `.release-epoch` (post-L3) — references the cached `ACCEPT_TEMP`'s resolved versions. A cache-hit on L3 could attempt to resolve a tarball GC just dropped.

Sources NOT to enumerate (DA round 2 answer): skill manifests under `~/.claude/skills/` (resolve via `npx`, different namespace), plugin lockfiles (IPFS, separate system), `CLAUDE.md` examples (illustrative, mostly `@latest` not pinned).

Related ADRs

- **ADR-0025** — persistent `npx` cache contract; this ADR's L9 extends the existing sweep to also LRU-prune `_cacache`.
- **ADR-0038** — `/tmp/ruflo-build` persistent build-source dir; this ADR's build-half scope-exclusions reference it.
- **ADR-0039** — `_run_and_kill` sentinel pattern; this ADR's L4 reparent move keeps the existing sentinel-file paths working (they live in `/tmp` and exit cleanly per the pattern).
- **ADR-0048** — persistent ONNX model cache (`~/.cache/agentdb-models`); this ADR does NOT touch it; the cache survives across releases by design.
- **ADR-0150** — napi rebuild gate; this ADR's `cross-repo/ruvector/target/` exclusion is deferred as scope-out because of the napi-rebuild dependency.

Measurement

Per-release measurement appends one JSON line each to two complementary logs:

logs/release-disk-bytes.jsonl (primary — SSD-wear proxy):

```
{"release": "patch.143", "write_bytes": 6312455168, "read_bytes": 14882017280, "duration_s": 408, "levers_active": ["L1", "L2", "L3", "L4"]}
```

logs/file-events.jsonl (secondary — fsevents-buffer proxy):

```
{"release": "patch.143", "events_write": 192340, "events_clonefile": 0, "events_read": 821100, "init_invocations": 17, "duration_s": 408, "levers_active": ["L1", "L2", "L3", "L4"]}
```

`init_invocations` is the count of distinct `init --full` calls per release across the pipeline (harness + per-check). Baseline: 17 (1 golden + 16 per-check across the acceptance corpus). Empirical per-init footprint (measured 2026-05-16 with `@sparkleideas/cli@latest`): **333 files / 2.6 MB** of project artifacts (`.claude/` 319 files / 2.5 MB; `.claude-flow/` 10 files / 52 KB; 4 top-level files; 12 directories). `init --full` **writes zero node_modules entries** — it does not invoke `npm install`, `pnpm install`, or any package manager (verified against the fork's `init.ts` and `init/mcp-generator.ts`; the only `npm` references in the source are user-facing instruction strings). The **50k node_modules files in `$ACCEPT_TEMP/node_modules` come exclusively from the harness's two `npm install` invocations against Verdaccio** (main install of `@sparkleideas/cli`, plus the wrapper solo install of `@sparkleideas/ruflo`); per-check checks reuse those installs via `$CLI_BIN` (direct binary call) or `_e2e_isolate`'s `node_modules` symlink, so per-check inits do not trigger additional `npm install` work. The persistent ONNX model cache at `~/cache/transformers` (~110 MB for `Xenova/all-mpnet-base-v2`, ADR-0048) survives across releases and is not part of the per-release init footprint. Multiplying the 333-file figure by 17 gives a per-release floor of ~5.7k project-artifact writes from inits alone. L11 + L12 + L13 land in this metric directly.

Hard release-end gate (the only auto-fail criterion): **`write_bytes` must not exceed $\max(\text{baseline} \times 1.10, \text{baseline} + 100 \text{ MB})$** at the lever's landing time. The 10% multiplier handles noise on large baselines; the `+ 100 MB` floor absorbs unrelated host activity for small post-cache-hit releases where 10% drops below the noise floor of a single browser tab. Event-count is tracked but not gated — by-design ~unchanged by L2 since clonefile syscalls still register.

Baseline (2026-05-16, pre-lever): not yet measured. **Baseline capture is a hard prerequisite to L6** — the first roadmap commit ("baseline-capture") records `write_bytes` over ≥ 2 baseline `npm run release` invocations, takes the median, and writes the value to `logs/release-disk-bytes.jsonl`'s first entry with `levers_active: []`. L6 does not land until that entry exists. The principle scales to L1/L3 where the baseline is the hard-gate comparison point — a missing baseline silently invalidates the gate.

Amendments

Amendment: Status reconciliation (2026-05-18) — partial implementation

Status kept `proposed` per the 2026-05-18 ADR status audit.

Landed (in `scripts/test-acceptance.sh`):

- **L2** — APFS clonefile snapshot (`scripts/test-acceptance.sh:463`).
- **L3** — persistent `ACCEPT_TEMP` via `_resolve_accept_temp` (:169, 253, 279, 293).
- **L4** — reparented per-check workdirs under trap-covered parent (:273).
- **L6** — trap-cover wrapper-solo install dir + P5 background init dir (:193, 195).
- **L7** — widened glob to cover all 7 prefix classes that orphan on cleanup (:223).
- **L9** — LRU-prune `_cacache` when total size exceeds soft cap (:795).
- `logs/release-disk-bytes.jsonl` exists.

Deferred / open:

- **L1** (sentinel-write pattern), **L5**, **L8**, **L10-L13**: no evidence in `scripts/` .
- **Baseline capture (the "baseline-capture" first roadmap commit)** — ADR text says "Baseline (2026-05-16, pre-lever): not yet measured." Without ≥ 2 baseline `npm run release` invocations recorded with `levers_active: []`, the hard release-end gate is

silently invalidated for L1/L3/L6.

- **Hard release-end gate** itself (the `write_bytes ≤ max(baseline × 1.10, baseline + 100 MB)` auto-fail criterion) — not wired into the pipeline.

Roughly half the levers shipped; the gate that makes the program self-policing has not. Reconciled as part of the 2026-05-18 status audit.